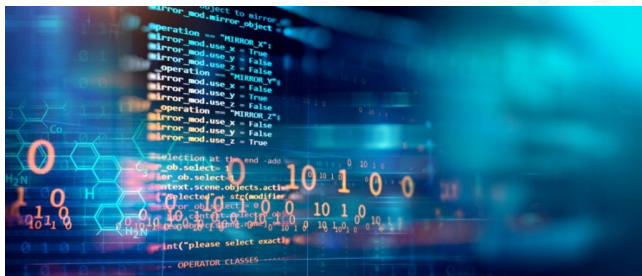


Programación Genérica

Problemas



Adolfo Muñoz – Juan Magallón
Grado en Ingeniería Informática



Universidad
Zaragoza



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

Sensores – Parte I

Problema



Una serie de sensores envían regularmente datos a un computador.

Los datos de cada sensor son procesados por distintos almacenes, cuya funcionalidad puede ser muy variada, pero que siempre ofrecen al menos estos métodos:

- enviar un dato:

```
store.push(v)
```

- enviar una serie de datos:

```
store.push( {a,b,c,d ...} )
```

- recuperar el valor guardado:

```
store << str.value()
```

Dado un tipo de dato (temperatura, velocidad) nos interesa poder guardar distintos almacenes para ese tipo de dato en una única estructura de datos.

La dificultad radica en los sensores pueden enviar datos de distintos tipos:

- Los sensores de tipo contador mandan números **enteros**.
- Los sensores analógicos trabajan con **reales** (pudiendo ser de simple o doble precisión).
- Ciertos sensores eléctricos y magéticos envían números **complejos**.
- Los sensores espectrales envían **vectores** de números reales.

Prueba tus almacenes de forma individual para distintos tipos de datos:

- `int`
- `double`
- `complex<float>`
- `vector<double>`

Problema



Diseña una clase StoreLast que simplemente

almacene el último dato

de los datos que va enviando un sensor.

Complicándolo un poco...

Problema



Diseña una clase StoreMax que

almacene el máximo

de todos los datos que va enviando un sensor.

Problema



Diseña una clase StoreMin que

almacene el mínimo

de todos los datos que va enviando un sensor.

Complicándolo un poco...

Problema



Diseña una clase StoreAvg que

almacene el valor medio

de todos los datos que va enviando un sensor,
pasándole un valor inicial (de cero) en el constructor.

Sensores – Parte II

Dado un sensor de un tipo concreto (real, complejo, ...), se desea hacer un seguimiento exhaustivo de sus diferentes estadísticas calculadas mediante diferentes acumuladores.

Específicamente:

- Deberá guardar distintos almacenes, especificados por el usuario.
- Ofrecerá un método `push( ??? v )` para actualizar todos los almacenes.
- Ofrecerá un método `log( )` que dará información del sensor.

Problema



Diseña una clase Logger a la que se le pase en el constructor una lista de acumuladores sobre el mismo sensor (mismo tipo de datos) pero que realicen diferentes tareas de acumulación.

Incluirá al menos dos métodos:

- `void push(??? v)`: recibe un valor y lo guarda en todos los almacenes.
- `void log()`: imprime la información del sensor.

Problema



Diseña una clase Logger ...

Puedes utilizar los contenedores estándar del lenguaje.

Para mejorar la información visualizada puedes añadir a las distintas clases alguna forma de identificarlas.

Ejemplo:

```
1  Logger<float> temperature_log( "temperature",
2      {
3      new StoreLast<float>,
4      new StoreMax<float>,
5      new StoreAvg<float>(0)
6      } );
7
8  temperature_log.push( 16.0 );
9  temperature_log.push( {17.5, 25.0, 20.5} );
10 temperature_log.log();
```

Ejemplo:

```
1 > ./main
2 temperature:
3     last: 20.5
4     max: 25
5     average: 19.75
```



Ejemplo:

```
1  Logger<complex<float>> orientation_log("orientation",
2      {
3          new StoreLast<complex<float>>,
4          new StoreAvg<complex<float>> (0)
5      } );
6
7  orientation_log.push( { {0,1.5}, {3.0}, {2.5,1.0} } );
8  orientation_log.log();
```

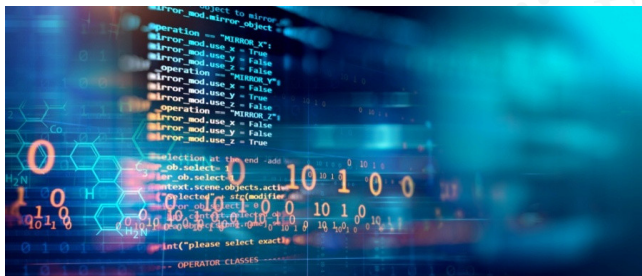
Ejemplo:

```
1 > ./main
2 orientation:
3     last: (2.5,1)
4     average: (1.83333,0.833333)
```



Programación Genérica

Problemas



Adolfo Muñoz – Juan Magallón
Grado en Ingeniería Informática



Universidad
Zaragoza



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza